GRUPO conforsa

ÁREA: PROGRAMACIÓN

PROGRAMACIÓN .NET(I)

Queda prohibida toda la reproducción de la obra o partes de la misma por cualquier medio sin la autorización previa

Área: Programación
Curso: Programación .Net(I)
Edición: Diciembre 2011

ÍNDICE

MÓDULO I - INTRODUCCIÓN A LA PROGRAMACIÓN

TEMA 1.- INTRODUCCIÓN

1.1. Definiciones e historia de la programación
1.2. Clasificación de lenguajes y Paradigmas de programación
1.3. Formalización de problemas y algoritmos

TEMA 2.- CONCEPTOS DE PROGRAMACIÓN ESTRUCTURADA

2.1. Introducción
2.2. Tipos de datos
2.3. Variables y operaciones
2.4. Estructuras de control
2.5. Subprogramas, procedimientos y funciones

TEMA 3.- CONCEPTOS DE PROGRAMACIÓN ORIENTADA A OBJETOS

3.1. Introducción
3.2. Abstracción
3.3. Encapsulación
3.4. Herencia
3.5. Polimorfismo y sobrecarga

MÓDULO II - INTRODUCCIÓN A .NET

TEMA 4.- PLATAFORMA .NET FRAMEWORK

4.1. Introducción
4.2. .Net Framework
4.3. Espacios de nombres y Ensamblados
4.4. Lenguajes soportados por .NET
4.5. Biblioteca de clases comunes

TEMA 5.- DESARROLLANDO EN .NET

5.1. Introducción
5.2. Desarrollo de aplicaciones .NET
5.3. Compiladores por línea de comandos
5.4. Entornos de programación

MÓDULO III - PROGRAMACIÓN BÁSICA CON VISUAL BASIC .NET

TEMA 6.- SINTAXIS BÁSICA

6.1. Introducción
6.2. Estructura de una aplicación Visual Basic
6.3. Variables, tipos de datos y operadores
6.4. Tipos complejos de datos (arrays y enumeraciones)

TEMA 7.- ESTRUCTURAS DE CONTROL, FUNCIONES Y PROCEDIMIENTOS

7.1. Introducción
7.2. Estructuras de control
7.3. Funciones y procedimientos

TEMA 8.- BUENAS PRÁCTICAS DE PROGRAMACIÓN

8.1. Introducción
8.2. Decálogo de buenas prácticas y comentarios en Visual Basic
8.3. Tratamiento de excepciones
8.4. Depuración y corrección de errores

MÓDULO IV - PROGRAMACIÓN ORIENTADA A OBJETOS CON VISUAL BASIC .NET

TEMA 9.- CLASES Y OBJETOS

- 6.1. Introducción
- 6.2. Clases y objetos
- 6.3. Miembros de una clase
- 6.4. Constructores y destructores

TEMA 10.- CARACTERÍSTICAS DE LA PROGRAMACIÓN ORIENTADA A OBJETOS

- 8.1. Introducción
- 8.2. Abstracción y Encapsulación
- 8.3. Herencia de Clases con Visual Basic
- 8.4. Polimorfismo con Visual Basic

TEMA 11.- INTERFACES Y COMPONENTES

- 8.1. Introducción
- 8.2. Interfaces
- 8.3. Eventos y delegados
- 8.4. Componentes .NET

TEMA 1.- INTRODUCCIÓN

1.1. Definiciones e historia de la programación

Actualmente el término programación está muy extendido y nos resulta familiar a la mayoría de nosotros por unas u otras razones. Sin embargo ¿que entendemos por programación?

La definición que encontramos en el Diccionario de la **R.A.E.** en su quinta acepción es:
"*Elaborar programas para la resolución de problemas mediante ordenadores.*"

Tal vez para intentar ser más técnicos podríamos decir que la programación consiste en:
"*Proporcionar instrucciones precisas (programas) a un computador (ordenador) para que lleve a cabo una serie de operaciones (resolución de problemas)*"

Otro termino que aparece relacionado cuando hablamos de programación es "**Lenguaje de programación**".

Cualquier lenguaje, incluso un lenguaje de programación, es un conjunto de signos y reglas que permiten la comunicación.

Por lo tanto si ampliamos un poco la definición que hemos hecho previamente podemos llegar a la conclusión de que la programación va a consistir en:

"El manejo de un lenguaje de programación que nos va a permitir comunicarnos con el ordenador para proporcionarle unas instrucciones precisas, con las que pueda llevar a cabo las operaciones necesarias para la resolución de un problema concreto"

Si nos fijamos en esta nueva definición nos daremos cuenta de que hemos dicho "el manejo de un lenguaje". ¿Por qué de un lenguaje? ¿Es que existen distintos lenguajes de programación? Como cabría esperar, la respuesta es **Sí**.

Los lenguajes de programación

Como habíamos dicho existen varios lenguajes de programación, y evidentemente son distintos, cada unos con sus características y particularidades, pero ¿Por qué existe esta disparidad de lenguajes?

Antes de responder a esta pregunta vamos a plantearnos un pequeño acertijo: ¿Qué tienen en común, la agenda de tu teléfono móvil, un video juego, un sistema de guiado de misiles, y la página Web de operaciones financieras de un banco? Efectivamente la respuesta es **PROGRAMACIÓN**.

Podemos decir que esta diversidad de lenguajes obedece a varias razones, desde motivos académicos hasta estrategias comerciales de grandes compañías de software, pero fundamentalmente debemos tener en cuenta:

- **El objetivo y ámbito de la aplicación a desarrollar**
- **El dispositivo sobre el que se va a programar**

Objetivo y ámbito de la aplicación

Existen lenguajes específicos o más enfocados a programar determinadas aplicaciones.

Siguiendo el ejemplo anterior para programar el sistema de guiado de un misil necesitaremos un lenguaje que sea fiable en la adquisición de datos y que pueda dar respuesta rápida, mientras que una aplicación bancaria necesitara un lenguaje cuyo punto fuerte sea la seguridad en la comunicación y las transacciones.

Por otro lado, para la programación de video juegos puede resultar útil un lenguaje que facilite la tarea de gestionar gráficos y animaciones.

El Dispositivo

Como veremos más adelante la evolución de los lenguajes de programación ha ido paralela al avance tecnológico en materia de hardware. Hasta hace muy poco los lenguajes disponibles estaban muy ligados al dispositivo sobre el que se quería programar, aunque la tendencia actual es intentar separar el lenguaje de programación de la máquina.

Si seguimos el ejemplo anterior es evidente que las capacidades de memoria y procesamiento de un teléfono móvil son muy distintas a las de un servidor bancario.

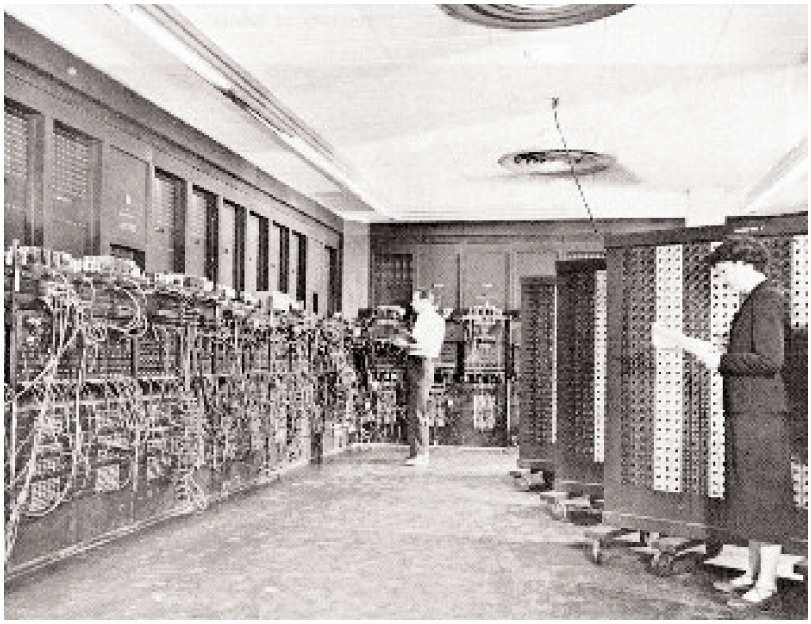
Historia de la programación

Para hablar de la evolución histórica de la programación y los lenguajes de programación es imprescindible hablar también de la evolución de los ordenadores desde máquinas para efectuar cálculos sencillos hasta los potentes ordenadores actuales capaces de gestionar, almacenar y transformar grandes cantidades de información.

Dentro de la historia de la computación es posible remontarse incluso a varios siglos antes de Cristo, y hacer referencia a la aparición del ábaco en China.

No es hasta el siglo XIX cuando aparecen los primeros conceptos teóricos, no solo sobre ordenadores sino sobre los lenguajes para manejar estas maquinas solo existentes de manera conceptual. Como nombres propios de esta época debemos destacar a Charles Babagge, un profesor y matemático de la universidad de Cambridge que desarrolló lo que él denominaba la maquina analítica, pero que por motivos técnicos no pudo construirse hasta mediados del siglo XX. Junto con Babagge hay que mencionar a Ada Lovedby, considerada como la primera mujer programadora de la historia, pues realizo programas para la maquina analítica de Babagge.

Sin embargo es a principios del siglo XX cuando empezamos a ver como la tecnología avanza de manera espectacular y podemos hablar de los primeros ordenadores.



Sin duda entre los primeros ingenios del siglo XX y que podemos considerar como los tatarabuelos de los ordenadores actuales se encuentra el ENIAC, construido en 1946, una maquina que ocupaba una habitación entera, pesaba 27 toneladas, utilizaba 1.500 conmutadores electromagnéticos y relés. Para su programación requería la operación manual de unos 6.000 interruptores.

El siguiente paso en la historia de la computación es la aparición de la maquina EDVAC que empezó a funcionar en 1951. Esta máquina se construyó con la idea de superar el rendimiento de ENIAC y para su diseño se contó con la ayuda de John Von Neumann, considerado como uno de los artífices de la arquitectura de computadores modernos.

La diferencia principal con su predecesora fue que no estaba diseñada para operar con números decimales sino con números binarios, como los ordenadores actuales, y tuvo el primer programa diseñado para ser almacenado.

Es a partir de este momento cuando los lenguajes de programación empiezan a cobrar importancia, pero habrá que esperar hasta 1953, con la entrada en el mercado de la computación industrial por parte de IBM, para ver como los lenguajes de programación cobran importancia, con la aparición del lenguaje ensamblador.

Otro gran hito en la historia de la programación es la aparición en los años 60 de los primeros lenguajes de alto nivel, más cercanos al usuario que a la máquina, como son COBOL y FORTRAN.

Pero la verdadera revolución de los lenguajes de programación viene aparejada a la introducción del ordenador en ámbitos más domésticos y sobre todo en medianas y pequeñas empresas gracias a los ordenadores personales, que conocemos vulgarmente hoy en día como PC. Es a mediados de la década de los 70 y sobre todo principios de los 80 cuando, gracias al abaratamiento y reducción de tamaño de los microprocesadores, el ordenador salta de las grandes corporaciones y laboratorios universitarios a las mesas de los despachos y oficinas y poco a poco hasta los hogares de muchas personas. Es esta gran dispersión en el tipo de usuario de ordenador así como las nuevas arquitecturas lo que favorece la aparición de nuevos lenguajes. Como ejemplo podemos citar el lenguaje Pascal y el conocidísimo lenguaje C.

A finales del siglo XX, en la década de los 90, el ordenador está presente en muchos de los aspectos de nuestra vida cotidiana y es esta accesibilidad la que se intenta llevar a los lenguajes de programación. Es precisamente en esta época cuando se potencian los lenguajes visuales, como Visual Basic, más acordes con los también renovados entornos de desarrollo que funcionan en sistemas operativos clasificados como WIMP (basados en Ventanas, Iconos, Menús y Puntero)

La expansión de Internet, y en general el uso de redes de computadores es otro de los aspectos influyentes en la aparición de nuevos lenguajes enfocados a satisfacer las nuevas necesidades como el desarrollo de aplicaciones Web, aplicaciones cliente-servidor o la necesidad de programación independiente de la plataforma. Podemos destacar lenguajes como PHP, o Java como exponentes de algunos de los lenguajes surgidos en estos tiempos.

Son estos cambios constantes, y cada vez más rápidos, en las tecnologías así como la necesidad de abordar problemas cada vez más complejos los que estimulan el avance de los lenguajes de programación existentes así como la aparición de lenguajes nuevos, siempre con la intención de dotar a los programadores de herramienta más potentes que posibiliten desarrollos más eficientes y fiables en el menor tiempo posible.

Con la intención de satisfacer estas nuevas necesidades apareció a principios del siglo XXI el entorno de desarrollo .NET . Esta tecnología es algo más que un simple lenguaje de programación, pero antes de entrar en profundidad en esta cuestión es necesario aclarar algunos conceptos previos.

1.2. Lenguajes y Paradigmas

Clasificación de lenguajes y paradigmas de programación

Entre tantos lenguajes de programación puede ser sencillo perderse, por eso es conveniente tener unas nociones básicas de las clasificaciones habituales en las que se enmarcan los lenguajes de programación y que nos pueden ayudar a la hora de decidirnos por el uso de un lenguaje u otro.

A la hora de clasificar un lenguaje podemos atender a los siguientes criterios:

- **Nivel de abstracción**
- **Forma de ejecución**
- **Paradigma de programación**

Algunos de estos criterios no son necesariamente excluyentes y podemos hablar por ejemplo de distintos tipos de paradigmas dentro de un mismo nivel de abstracción.

Nivel de abstracción

El nivel de abstracción nos indica lo cercano que está el lenguaje a instrucciones que entiende el computador, o lo cercano que se encuentra al lenguaje natural que entiende el ser humano. Según esta clasificación podemos hablar de:

- **Lenguaje maquina**. Es el más cercano al ordenador y consiste en cargar en memoria la secuencia binaria de ceros y unos que representan cada instrucción
- **Lenguaje de bajo nivel**. Sería el siguiente paso al lenguaje maquina. Aunque se sustituyen las cadenas binarias por palabras más comprensibles (denominadas nemónico) su funcionamiento se basa en dar instrucciones directas al microprocesador. El lenguaje de bajo nivel por excelencia es el lenguaje ensamblador
- **Lenguaje de medio nivel**. Posee características de los lenguajes de bajo nivel y de los de alto nivel. El lenguaje C es un claro exponente de este tipo de lenguajes.
- **Lenguajes de alto nivel**. Son los más cercanos al lenguaje natural, como pudiera ser el español o el inglés. Están mas centrados en la resolución del problema que en como debemos dar las instrucciones a la maquina. Un claro ejemplo de este tipo de lenguajes pueden ser Pascal o Basic

Ejemplo de distinto nivel de abstracción

Ejemplo de lenguajes de diferente nivel:

Binario:

10110000 01100001 (Hexadecimal: 0xb061)

Lenguaje ensamblador:

MOV al, 061h
Esta instrucción significa:
Mueva el valor hexadecimal 61 (97 decimal) al registro "al".

Medio nivel:

```
bool resultado = (x!=5)?true:false;
```

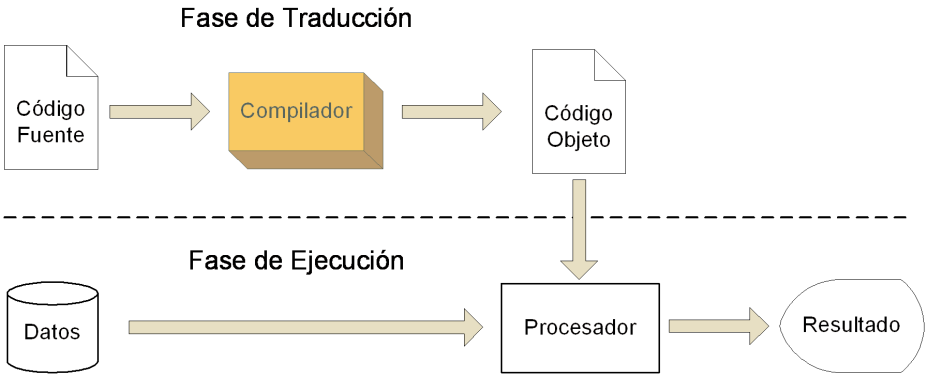
Alto nivel:

```
If (x=5) then
Begin
        Writeln("x es igual a 5");
        Readln;
end
.
```

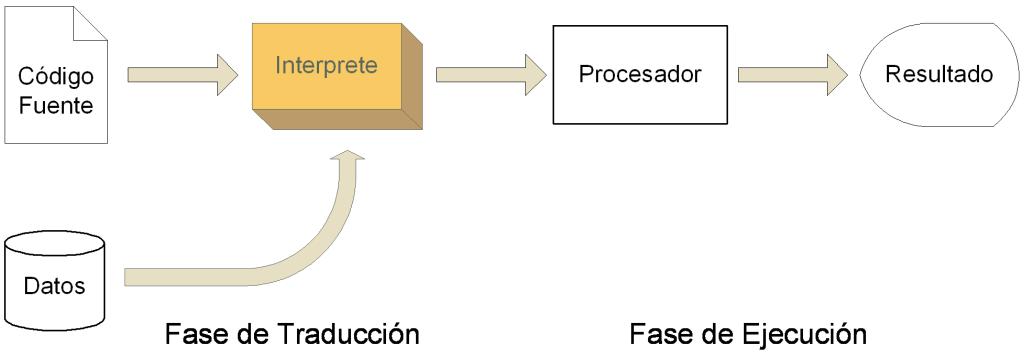
Forma de ejecución

Cuando se está trabajando con lenguajes de alto nivel llega un momento en que es necesario traducir las instrucciones de nuestro programa a un lenguaje que entienda el ordenador. Este lenguaje es el lenguaje maquina por lo tanto según se haga esta traducción podemos encontrar:

- **Lenguajes compilados**. Son aquellos que se sirven de un compilador para convertir las sentencias, todas de una vez siempre que el programa no contenga errores, en código entendible por el ordenador. Este código compilado, denominado código objeto, puede volver a ser almacenado y ejecutado de nuevo las veces que queramos sin necesidad de volver a compilar nuestro programa.



- **Lenguajes interpretados**. Son aquellos que se sirven de un intérprete para traducir las instrucciones de nuestro programa a un lenguaje entendible por el ordenador. A diferencia de un compilador, el interprete no genera ningún código objeto sino que convierte cada proposición del programa fuente en lenguaje máquina conforme vaya siendo necesario durante el procesamiento de los datos.



Paradigma de programación

Un paradigma no es más que una manera de hacer las cosas. Desde el punto de vista de programación, es el enfoque que vamos tomar a la hora de hacer los programas. Existen multitud de paradigmas, pero por mencionar algunos de los más extendidos podemos citar:

- **Programación estructurada**
- **Programación funcional**
- **Programación lógica**
- **Programación orientada a objetos**

Lo que de momento debes tener claro sobre estos paradigmas es que no son unos mejores o peores que otros, sino simplemente diferentes, aunque como cabría esperar algunos son mas apropiados que otros para la resolución de problemas concretos.

A lo largo del curso nos vamos a centrar en la programación estructurada y en la programación orientada a objetos, como los referentes de programación elementales que debes conocer.

Programación estructurada

La programación se divide en bloques (procedimientos y funciones) que pueden o no comunicarse entre sí. Además la programación se controla con secuencia, selección e iteración. Permite reutilizar código programado y otorga una mejor compresión de la programación.

Programación funcional

Este paradigma concibe a la computación como la evaluación de funciones matemáticas y evita declarar y cambiar datos. En otras palabras, hace hincapié en la aplicación de las funciones y composición entre ellas, más que en los cambios de estados y la ejecución secuencial de comandos (como lo hace el paradigma procedimental).

Permite resolver ciertos problemas de forma elegante y los lenguajes puramente funcionales evitan los efectos secundarios comunes en otro tipo de programaciones.

Programación lógica

Se basa en la definición de reglas lógicas para luego, a través de un motor de inferencias lógicas, responder preguntas planteadas al sistema y así resolver los problemas.

Programación orientada a objetos

Está basado en la idea de encapsular estado y operaciones en objetos. En general, la programación se resuelve comunicando dichos objetos a través de mensajes (programación orientada a mensajes)

Su principal ventaja es la reutilización de códigos y su facilidad para pensar soluciones a determinados problemas.

1.3. Formalización de problemas y algoritmos

No debes olvidar que el objetivo de la programación es la resolución de problemas por medio de un ordenador y que el objetivo de aprender un lenguaje es poder comunicarte con el ordenador para que realice las tareas necesarias para la resolución de dichos problemas.

Es muy frecuente la aplicación de ordenadores a problemas que requieran llevar a cabo tareas repetitivas o cálculos matemáticos complejos.

Para ilustrar más fácilmente como abordar un problema que podemos convertir en un programa tomemos el siguiente ejemplo: "Sumar todos los números entre el 0 y el 100 ". Ciertamente es una tarea repetitiva, aunque abordable, y evidentemente el cálculo matemático es sencillo, sin embargo algo tan sencillo como esto que puede llevarnos unos cuantos minutos con una calculadora y un papel se convierte en un problema de resolución trivial cuando se emplea programación.

Lo primero que debes hacer al enfrentarte a un problema es formalizar los pasos a dar para llegar a su correcta resolución. Esta lista bien definida, ordenada y finita de operaciones que permite hallar la solución a un problema es lo que se conoce con el nombre de algoritmo.

Existen múltiples herramientas para la formalización de algoritmos, pero son esenciales el conocimiento de dos de ellas:

- Diagrama de flujo
- Pseudocódigo

Diagrama de flujo

Un diagrama de flujo es la representación gráfica de una secuencia de acciones que permiten especificar los detalles de un proceso a través de la esquematización y la utilización de unos símbolos sencillos y concretos.

Se denomina de flujo porque entre los símbolos utilizados se encuentran las flechas, que van a indicar la secuencia que llevarán las distintas operaciones.

Los símbolos más comúnmente utilizados son:

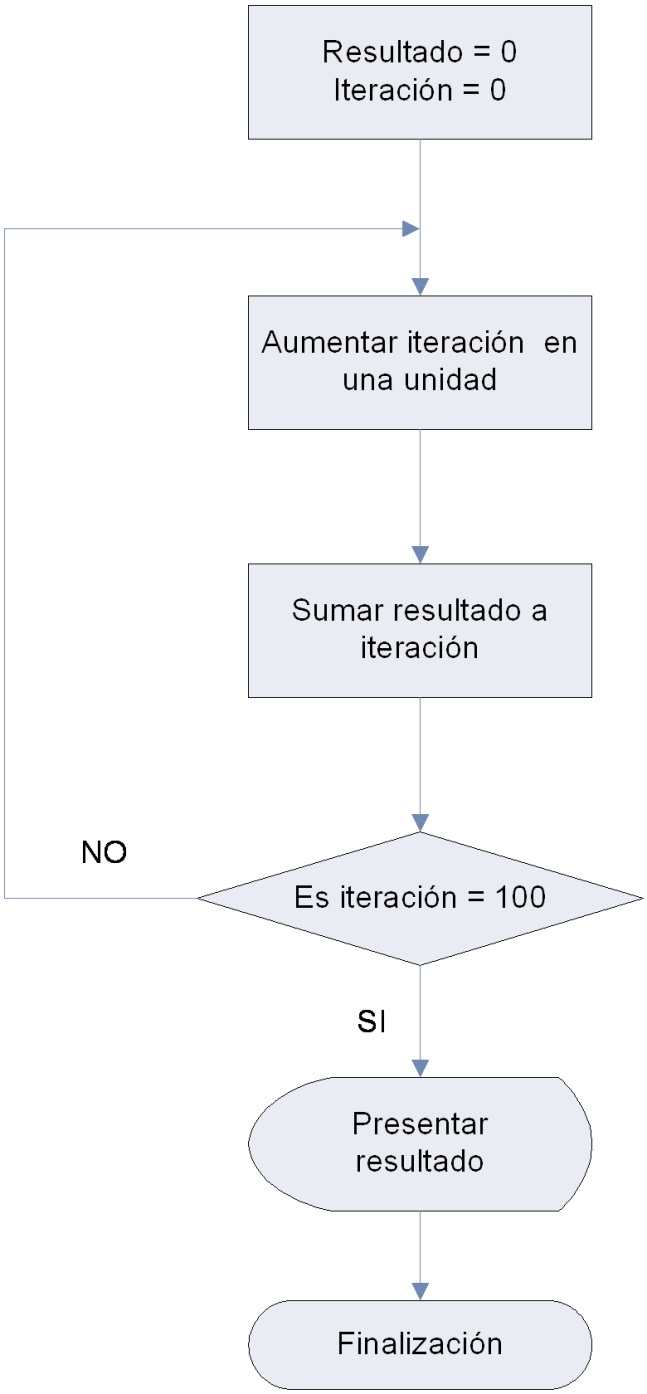
- **Rectángulo**. Representa un evento o proceso determinado que sucede de forma automática.
- **Rombo**. Representa una condición o bifurcación en el flujo de información, el cual entra por arriba y sale por un lado si la condición se cumple o sale por el lado opuesto si la condición no se cumple.
- **Círculo**. Re presenta un punto de conexión entre procesos. Se utiliza cuando es necesario dividir un diagrama de flujo en varias partes por razones de simplicidad
- **Flecha**. Indica el sentido del proceso de información o tarea.

A la hora de diseñar un diagrama de flujo debemos tener en cuenta que el proceso a modelar debe contar con un punto de entrada y otro de salida y que debe existir al menos un camino que conecte ambos puntos.

También es conveniente tener claro cuales son los objetivos que pretendemos conseguir, es decir cuales son los limites del asunto que se está modelando, así como el nivel de detalle al que queremos llegar especificando los procesos.

Utilizando hábilmente esta herramienta podrás identificar más fácilmente los problemas descubriendo los pasos redundantes, los cuellos de botella, sumideros de información, etc... y te resultará mas sencillo describir una solución adecuada.

Puedes ver como se puede modelar el problema del ejemplo anterior, "sumar los números del 0 al 100" utilizando la herramienta de flujo de datos.



Pseudocódigo

El pseudocódigo es otra interesante herramienta para la descripción de algoritmos. Consiste en especificar un problema mediante el uso del lenguaje natural, pero siguiendo algunas pautas y modelos que lo asemejan a un lenguaje de programación.

Puede que está definición pueda parecer un poco abstracta pero aunque no lo creas seguramente has manejado pseudocódigo con anterioridad sin ni siquiera saberlo.

Ejemplo de pseudocódigo

```
Desde 1 hasta 10
        Extraer  una patata del saco
Pelar las patatas
Cortar las patatas en dados
Poner aceite en la sartén
Calentar sartén
Mientras el aceite no este caliente
        Esperar
Poner  las patatas en la sartén
Mientras las patatas no estén doradas
        Esperar
Poner sal
Si las patatas están sosas
        Volver a poner sal
```

Una de las ventajas, que a su vez puede considerarse un inconveniente, es que el pseudocódigo no está estandarizado, es decir no existe unas reglas para la creación de pseudocódigo por lo que puedes ajustarlo al nivel de detalle.

Otras ventajas del pseudocódigo es que se puede describir algoritmos en menos espacio que con un diagrama de flujo y además, al parecerse a un lenguaje de programación, puede utilizarse como punto de partida para el desarrollo de un programa.

Es muy interesante comprobar como queda el ejemplo de la suma de números descrito en pseudocódigo

Ejemplo de suma de números

```
Suma vale 0
Iteración vale 0
Repetir
        Aumentar valor de Iteración en una unidad
        Suma vale (valor actual de Suma) + Iteración
Hasta que valor de Iteración sea igual a 100
Presentar valor de Suma
```

Ahora que ya conoces algo sobre algoritmos y algunos métodos para afrontar su resolución debes tener en cuenta que no todos los problemas pueden ser resueltos mediante un algoritmo. La teoría de la computabilidad es una rama de las ciencias de la computación que se encarga de estudiar la posibilidad o no de resolver un cierto problema a través de un algoritmo.

De todas formas, aunque un problema pueda resolverse también hay que tener en cuenta los recursos necesarios para llevarlo a cabo. La teoría de la complejidad computacional se centra en el estudio teórico de los recursos, expresados generalmente en tiempo y memoria, necesarios para alcanzar la resolución de un problema con un algoritmo.

Como divertimento te planteo tres problemas famosos que han sido abordados desde el punto de vista de la algoritmia. Tal vez al final del curso seas capaz de resolverlos empleando las técnicas de programación que hayas adquirido.

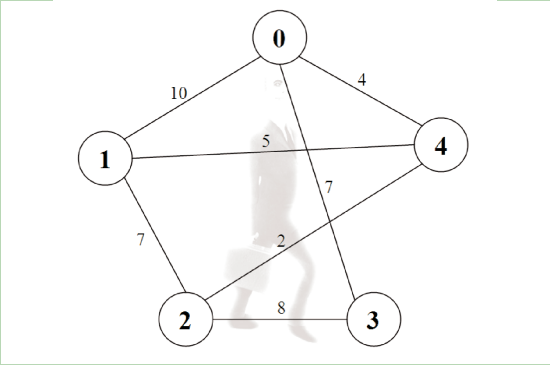
Ordenación por el método de la burbuja

Dada una lista de **n** números, ordena la lista de menor a mayor, intercambiándolos de posición si están en el orden equivocado. Es necesario revisar varias veces toda la lista hasta que no se necesiten más intercambios, lo cual significará que la lista está ordenada

Las torres de Hanoi

El juego consiste en pasar todos los discos a la tercera varilla colocados de mayor a menor ascendentemente.
Las reglas son:

1. Sólo se puede mover un disco cada vez
2. Un disco de mayor tamaño no puede descansar sobre uno más pequeño que él mismo.
3. Sólo puedes desplazar el disco que se encuentre arriba en cada varilla.

Las torres de Hanoi

Sean **n** ciudades de un territorio. El objetivo es encontrar una ruta que, comenzando y terminando en una ciudad concreta, pase una sola vez por cada una de las ciudades y minimice la distancia recorrida por el viajante